Js code

JavaScript Operator Hierarchy — High → Low

Priority	Operators	Associativity
1	`()` `[]` `.` `?.`	Left → Right
2	`++` `--` `!` `~` `+` `-` `typeof` `void` `delete`	**Right → Left**
3	`**`	**Right → Left**
4	`*` `/` `%`	Left → Right
5	`+` `-`	Left → Right
6	`<<` `>>` `>>>`	Left → Right
7	`<` `<=` `>` `>=` `in` `instanceof`	Left → Right
8	`==` `!=` `===` `!==`	Left → Right
9	`&`	Left → Right

9	`&`	Left → Right
10	`^`	Left → Right
11	`	`
12	`&&`	Left → Right
13	`	
14	`?:`	**Right → Left**
15	`=` `+=` `-=` `*=` `/=` `%=` etc.	**Right → Left**
16	`,`	Left → Right

Execution Context kya hai?

JavaScript mein **Execution Context** ek aisa **environment** hai jisme JavaScript code ko **execute** kiya jata hai.

Simple language mein:

> **Execution Context = JavaScript ko code run karne ke liye milne wala environment.**

Is environment mein JS ko pata hota hai:

- variables/functions kahan hain
- `this` kis object ko refer karega
- outer scope kahan hai
- code ko kaise execute karna hai

1. Execution Context ke types

JavaScript mein mainly **3 types** samjho:

Type	Kab banta hai?
Global Execution Context	JavaScript file/script start hote hi
Function Execution Context	Jab function call hota hai
Eval Execution Context	`eval()` use karne par

Interview aur practical JS mein mainly **Global + Function Execution Context** important hain.

2. Global Execution Context

Jab JS file start hoti hai:

JavaScript

```javascript
var name = "Tarun";

function hello() {
    console.log("Hello");
}

console.log(name);
```

Sabse pehle **Global Execution Context** create hota hai.

Conceptually:

```
Global Execution Context
│
├── Variables
│   └── name
│
├── Functions
│   └── hello
│
├── this
│
└── Outer Environment
```

3. Function Execution Context

Jab function call hota hai:

JavaScript

```
function add(a, b) {
    let result = a + b;
    return result;
}

add(10, 20);
```

`add()` call hote hi ek **new Function Execution Context** create hota hai.

`add()` call hote hi ek **new Function Execution Context** create hota hai.

```
Global Execution Context
        |
        | add(10, 20)
        ↓
Function Execution Context
        |
        ├── a = 10
        ├── b = 20
        ├── result = 30
        └── this
```

Function complete hone ke baad uska execution context **remove** ho jata hai.

4. Execution Context ke andar kya hota hai?

Execution Context ko simple way mein 3 important parts se samjho:

```
Execution Context
|
├── Variable Environment
|
├── Lexical Environment
|
└── This Binding
```

Js code

```javascript
let x = 10;

function test() {
    let y = 20;
    console.log(x + y);
}

test();
```

`test()` ke context mein:

```
Function Execution Context
│
├── y = 20
├── x → outer/global scope se milega
└── this
```

Isi **lexical environment / scope chain** ki wajah se function outer variables ko access kar sakta hai.

Haan, **conceptually JavaScript execution ko 2 phases mein samjha jata hai**:

```
JavaScript Execution
        ↓
┌──────────────────────┐
│ 1. Creation Phase    │
└──────────────────────┘
        ↓
┌──────────────────────┐
│ 2. Execution Phase   │
└──────────────────────┘
```

1. Creation Phase

Is phase mein JavaScript **execution context prepare** karta hai.

Example:

JavaScript

```
var a = 10;

function add() {
    console.log("Hello");
}
```

Creation phase mein conceptually:

```
a       → undefined
add     → function definition
this    → set
scope   → setup
```

Isliye `var` hoisting possible hoti hai.

2. Execution Phase

Ab JavaScript code ko **line-by-line execute** karta hai.

JavaScript

```
var a = 10;

function add() {
    console.log("Hello");
}

console.log(a);
add();
```

Execution:

```
a = 10
     ↓
console.log(a) → 10
     ↓
add() call
     ↓
"Hello"
```

Js code

```
// 1. String
let name = "Tarun";

console.log(typeof name);
// Output: string



// 2. Number
let age = 22;

console.log(typeof age);
// Output: number



// 3. Number - Decimal
let price = 99.99;

console.log(typeof price);
// Output: number
```

Js code

```javascript
// 4. BigInt
let bigNumber = 123456789012345678901234567890n;

console.log(typeof bigNumber);
// Output: bigint



// 5. Boolean
let isStudent = true;

console.log(typeof isStudent);
// Output: boolean



// 6. Undefined
let x;

console.log(typeof x);
// Output: undefined
```

Js code

JavaScript

```javascript
// 7. Null
let data = null;

console.log(typeof data);
// Output: object
// ⚠️ JavaScript ka historical behavior hai
// typeof null → "object"



// 8. Symbol
let id = Symbol("id");

console.log(typeof id);
// Output: symbol



// 9. Object
let user = {
    name: "Tarun",
    age: 22
};
```

Js code

```
console.log(typeof user);
// Output: object



// 10. Array
let skills = ["C++", "JavaScript", "Node"];

console.log(typeof skills);
// Output: object

// ⚠️ Array ka typeof bhi object hota hai



// 11. Function
function greet() {
    console.log("Hello");
}

console.log(typeof greet);
// Output: function
```

Js code

```
// 12. Date Object
let date = new Date();

console.log(typeof date);
// Output: object



// 13. RegExp Object
let pattern = /hello/;

console.log(typeof pattern);
// Output: object
```

Js code

```
const user = {
    name: "Tarun",
    age: 22,
    city: "Indore"
};


// ================================================
// 1. Dot Notation
// ================================================

console.log(user.name);
// Output: Tarun



// ================================================
// 2. Bracket Notation
// ================================================

console.log(user["name"]);
// Output: Tarun
```

Js code

```
// 3. Dynamic Key using Variable
// ========================================================

const key = "name";

console.log(user[key]);
// Output: Tarun



// ========================================================
// 4. Object.keys()
// ========================================================

console.log(Object.keys(user));
// Output: [ 'name', 'age', 'city' ]

// name ko find karna
console.log(Object.keys(user)[0]);
// Output: name
```

Js code

```
// 5. Object.values()
// ================================================

console.log(Object.values(user)[0]);
// Output: Tarun



// ================================================
// 6. Object.entries()
// ================================================

console.log(Object.entries(user)[0]);
// Output: [ 'name', 'Tarun' ]
```

Js code

```
// 7. Object.entries() + Loop
// ========================================================

for (const [key, value] of Object.entries(user)) {
    if (key === "name") {
        console.log(value);
    }
}

// Output: Tarun



// ========================================================
// 8. for...in
// ========================================================

for (const key in user) {
    if (key === "name") {
        console.log(user[key]);
    }
}
```

Js code

JavaScript

```
// 9. hasOwnProperty()
// ========================================================

console.log(user.hasOwnProperty("name"));
// Output: true




// ========================================================
// 10. Object.hasOwn()
// ========================================================

console.log(Object.hasOwn(user, "name"));
// Output: true




// ========================================================
// 11. in Operator
// ========================================================

console.log("name" in user);
// Output: true
```

Js code

```
// 12. Destructuring
// ========================================================

const { name } = user;

console.log(name);
// Output: Tarun



// ========================================================
// 13. Destructuring with Different Variable Name
// ========================================================

const { name: userName } = user;

console.log(userName);
// Output: Tarun
```

2. Object with Array

Object ke andar array:

JavaScript

```
const user = {
    name: "Tarun",
    skills: ["C++", "JavaScript", "Node.js"]
};

console.log(user.name);
// Output: Tarun

console.log(user.skills);
// Output: [ 'C++', 'JavaScript', 'Node.js' ]

console.log(user.skills[0]);
// Output: C++

console.log(user.skills[1]);
// Output: JavaScript
```

3. Array of Objects ⭐

Ye **bahut important** hai.

JavaScript

```javascript
const users = [
    {
        name: "Tarun",
        age: 22
    },
    {
        name: "Rahul",
        age: 23
    },
    {
        name: "Aman",
        age: 21
    }
];

console.log(users[0]);
// Output: { name: 'Tarun', age: 22 }

console.log(users[0].name);
```

Js code

4. Nested Object

Object ke andar object = nested object.

JavaScript

```javascript
const user = {
    name: "Tarun",

    address: {
        city: "Indore",
        state: "Madhya Pradesh"
    }
};

console.log(user.name);
// Output: Tarun

console.log(user.address);
// Output: { city: 'Indore', state: 'Madhya Pradesh' }

console.log(user.address.city);
// Output: Indore

console.log(user.address.state);
```

5. Embedded Object

Embedded object ka matlab generally parent object ke andar related object ko directly store karna.

Example:

JavaScript

```
const student = {
    name: "Tarun",

    address: {
        city: "Indore",
        pincode: 453441
    },

    education: {
        college: "Medi-Caps",
        branch: "CSE"
    }
};

console.log(student.address.city);
// Output: Indore
```

Js code

6. Object → Array → Objects ⭐

Real projects me ye bahut common hai.

JavaScript

```javascript
const company = {
    name: "ABC Company",

    employees: [
        {
            name: "Tarun",
            age: 22
        },
        {
            name: "Rahul",
            age: 23
        }
    ]
};

console.log(company.name);
// Output: ABC Company

console.log(company.employees);
```

Js code

```
console.log(company.name);
// Output: ABC Company

console.log(company.employees);
// Output:
// [
//   { name: 'Tarun', age: 22 },
//   { name: 'Rahul', age: 23 }
// ]

console.log(company.employees[0].name);
// Output: Tarun

console.log(company.employees[1].name);
// Output: Rahul
```

Js code

7. Object → Object → Array

JavaScript

```javascript
const user = {
    name: "Tarun",

    address: {
        city: "Indore",

        nearbyPlaces: [
            "Ujjain",
            "Dewas",
            "Bhopal"
        ]
    }
};

console.log(user.address.city);
// Output: Indore

console.log(user.address.nearbyPlaces[0]);
// Output: Ujjain

console.log(user.address.nearbyPlaces[2]);
```

Js code

```
const response = {
    success: true,

    user: {
        name: "Tarun",

        address: {
            city: "Indore"
        },

        skills: [
            "JavaScript",
            "Node.js",
            "MongoDB"
        ],

        projects: [
            {
                name: "Vehicle Rental",
                tech: ["React", "Node.js", "MongoDB"]
            },

            {
```

Js code

```
console.log(response.success);
// Output: true

console.log(response.user.name);
// Output: Tarun

console.log(response.user.address.city);
// Output: Indore

console.log(response.user.skills[0]);
// Output: JavaScript

console.log(response.user.projects[0].name);
// Output: Vehicle Rental

console.log(response.user.projects[0].tech[1]);
// Output: Node.js
```

Js code

JavaScript Object Methods

Share

S.No.	Method	Kya karta hai	Example	Output
1	`Object.keys()`	Saari keys deta hai	`Object.keys(user)`	`["name","age","city"]`
2	`Object.values()`	Saari values deta hai	`Object.values(user)`	`["Tarun",22,"Indore"]`
3	`Object.entries()`	Key-value pairs deta hai	`Object.entries(user)`	`[["name","Tarun"],...]`
4	`Object.fromEntries()`	Entries ko object banata hai	`Object.fromEntries(entries)`	`{name:"Tarun"}`
5	`Object.assign()`	Objects ko merge/copy karta hai	`Object.assign({}, user)`	Copy of object
6	`Object.hasOwn()`	Own property check karta hai	`Object.hasOwn(user,"name")`	`true`
7	`Object.create()`	New object create karta hai	`Object.create(obj)`	New object

Share

7	`Object.create()`	New object create karta hai	`Object.create(obj)`	New object
8	`Object.freeze()`	Object ko modify hone se rokta hai	`Object.freeze(user)`	Frozen object
9	`Object.isFrozen()`	Frozen hai ya nahi check	`Object.isFrozen(user)`	`true/false`
10	`Object.seal()`	Add/delete properties rokta hai	`Object.seal(user)`	Sealed object
11	`Object.isSealed()`	Sealed hai ya nahi	`Object.isSealed(user)`	`true/false`
12	`Object.preventExtensions()`	New property add karna rokta hai	`Object.preventExtensions(user)`	Object
13	`Object.isExtensible()`	New properties add ho sakti hain?	`Object.isExtensible(user)`	`true/false`

Js code

	`e()`	ho sakti hain?		Share •••
14	`Object.getOwnPrope rtyNames()`	Own property names deta hai	`Object.getOwnPropertyNames (user)`	`["name","age","city"]`
15	`Object.getOwnPrope rtySymbols()`	Own Symbol properties deta hai	`Object.getOwnPropertySymbo ls(user)`	`[Symbol(...)]`
16	`Object.getOwnPrope rtyDescriptor()`	Property ki details deta hai	`Object.getOwnPropertyDescr iptor(user,"name")`	Descriptor object
17	`Object.getOwnPrope rtyDescriptors()`	All property descriptors deta hai	`Object.getOwnPropertyDescr iptors(user)`	Descriptor object
18	`Object.definePrope rty()`	Property create/modify karta hai	`Object.defineProperty(... )`	Object
19	`Object.definePrope rties()`	Multiple properties defin ta hai	`Object.defineProperties(.. .)`	Object

19	`Object.definePrope rties()`	Multiple properties define karta hai	`Object.defineProperties(.. .)`	Object
20	`Object.getPrototyp eOf()`	Prototype deta hai	`Object.getPrototypeOf(user )`	Prototype
21	`Object.setPrototyp eOf()`	Prototype set karta hai	`Object.setPrototypeOf(a,b )`	Object
22	`Object.is()`	Do values ko accurately compare karta hai	`Object.is(10,10)`	`true`

Js code

Ek hi code me important methods

JavaScript

```
const user = {
    name: "Tarun",
    age: 22,
    city: "Indore"
};

console.log(Object.keys(user));
// Output: [ 'name', 'age', 'city' ]

console.log(Object.values(user));
// Output: [ 'Tarun', 22, 'Indore' ]

console.log(Object.entries(user));
// Output: [ [ 'name', 'Tarun' ], [ 'age', 22 ], [ 'city', 'Indore' ] ]

const entries = [["name", "Tarun"], ["age", 22]];

console.log(Object.fromEntries(entries));
// Output: { name: 'Tarun', age: 22 }
```

Js code

```
const entries = [["name", "Tarun"], ["age", 22]];

console.log(Object.fromEntries(entries));
// Output: { name: 'Tarun', age: 22 }

console.log(Object.hasOwn(user, "name"));
// Output: true

console.log(Object.hasOwn(user, "salary"));
// Output: false

const copy = Object.assign({}, user);

console.log(copy);
// Output: { name: 'Tarun', age: 22, city: 'Indore' }

console.log(Object.is(user, user));
// Output: true
```

1. Normal `for` loop ⭐

Sabse basic loop.

JavaScript

```
const arr = ["C++", "JavaScript", "Node"];

for (let i = 0; i < arr.length; i++) {
    console.log(arr[i]);
}

// Output:
// C++
// JavaScript
// Node
```

Use kab?

Jab **index bhi chahiye**.

JavaScript

```
console.log(arr[0]);
```

2. `for...of` ⭐⭐⭐

Array ke **values** directly milti hain.

JavaScript

```
const arr = ["C++", "JavaScript", "Node"];

for (const value of arr) {
    console.log(value);
}

// Output:
// C++
// JavaScript
// Node
```

Index bhi chahiye?

JavaScript

```
for (const [index, value] of arr.entries()) {
    console.log(index, value);
}

// Output:
// 0 C++
// 1 JavaScript
// 2 Node
```

Golden rule:

```
for...of → VALUES
```

3. `for...in` ⭐⭐⭐

Object ki **keys** ke liye commonly use hota hai.

JavaScript

```
const user = {
    name: "Tarun",
    age: 22,
    city: "Indore"
};

for (const key in user) {
    console.log(key);
}

// Output:
// name
// age
// city
```

Value chahiye:

Value chahiye:

JavaScript

```
for (const key in user) {
    console.log(user[key]);
}

// Output:
// Tarun
// 22
// Indore
```

Key + value:

JavaScript

```
for (const key in user) {
    console.log(key, user[key]);
}

// Output:
// name Tarun
// age 22
```

4. `forEach()` ⭐⭐⭐

Array ka **har element** process karta hai.

JavaScript

```
const arr = ["C++", "JavaScript", "Node"];

arr.forEach(function(value) {
    console.log(value);
});

// Output:
// C++
// JavaScript
// Node
```

Arrow function:

JavaScript

```
arr.forEach((value) => {
    console.log(value);
});
```

`forEach()` me index

JavaScript

```
arr.forEach((value, index) => {
    console.log(index, value);
});

// Output:
// 0 C++
// 1 JavaScript
// 2 Node
```

Parameters ka order:

JavaScript

```
(value, index, array)
```

a2ff-79dd3b562cd0

Parameters ka order:

JavaScript

```
(value, index, array)
```

Example:

JavaScript

```
arr.forEach((value, index, array) => {
    console.log(value);
    console.log(index);
    console.log(array);
});
```

5. `while` loop

Jab tak condition `true` hai, loop chalega.

JavaScript

```
let i = 0;

while (i < 3) {
    console.log(i);
    i++;
}

// Output:
// 0
// 1
// 2
```

6. `do...while`

Isme **minimum ek baar** code execute hota hai.

JavaScript

```
let i = 5;

do {
    console.log(i);
    i++;
} while (i < 3);

// Output:
// 5
```

Condition initially false thi, fir bhi ek baar execute hua.

Js code

Array

JavaScript

```
const arr = ["A", "B", "C"];

for (const value of arr) {
    console.log(value);
}

// Output:
// A
// B
// C
```

`for...in`:

JavaScript

```
for (const index in arr) {
    console.log(index);
}

// Output:
// 0
```

Object ko `Object.entries()` ke saath use karo:

JavaScript

```
for (const [key, value] of Object.entries(user)) {
    console.log(key, value);
}

// Output:
// name Tarun
// age 22
```

9. `Object.keys()` + `for...of`

JavaScript

```javascript
const user = {
    name: "Tarun",
    age: 22,
    city: "Indore"
};

for (const key of Object.keys(user)) {
    console.log(key);
}

// Output:
// name
// age
// city
```

Js code

JavaScript

```
for (const key of Object.keys(user)) {
    console.log(user[key]);
}

// Output:
// Tarun
// 22
// Indore
```

---

10. `Object.values()` + `for...of`

JavaScript

```
for (const value of Object.values(user)) {
    console.log(value);
}

// Output:
// Tarun
```

> `this` ki value function kaise call hua hai, us par depend karti hai. Arrow function apna `this` nahi banata; surrounding `this` leta hai.

---

1. Object ke andar `this` ⭐⭐⭐

JavaScript

```
const user = {
    name: "Tarun",
    age: 22,

    show: function () {
        console.log(this.name);
        console.log(this.age);
    }
};

user.show();

// Output:
// Tarun
// 22
```

Js code

JavaScript

```
user.show()
```

`show()` ko `user` object ke through call kiya gaya.

Isliye:

JavaScript

```
this === user
```

```
user.show()
    ↓
  this
    ↓
  user
```

Js code

2. Object me `this` se property access

JavaScript

```
const user = {
    name: "Tarun",

    greet: function () {
        console.log("Hello " + this.name);
    }
};

user.greet();

// Output:
// Hello Tarun
```

`this.name` ka matlab:

JavaScript

```
user.name
```

Js code

3. Object me arrow function ⚠️

JavaScript

```
const user = {
    name: "Tarun",

    greet: () => {
        console.log(this.name);
    }
};

user.greet();
```

Output generally:

```
undefined
```

Kyunki arrow function ka **apna** `this` **nahi hota.**

```
user.greet()
      ↓
Arrow function
      ↓
apna this ❌
      ↓
surrounding this
```

Isliye object ke direct method ke liye generally:

JavaScript

```
greet() {
    console.log(this.name);
}
```

ya normal function use karna better hai.

Js code

4. Object + nested function ⭐

JavaScript

```
const user = {
    name: "Tarun",

    show: function () {

        function inner() {
            console.log(this.name);
        }

        inner();
    }
};

user.show();
```

Yaha `inner()` normal function hai.

`inner()` ko `user.inner()` ke form me call nahi kiya gaya.

Isliye `inner` ka `this`, `show` ka `this` automatically nahi leta.

5. Object + Arrow Function ⭐⭐

Yahi jagah arrow function useful hai:

JavaScript

```
const user = {
    name: "Tarun",

    show: function () {

        const inner = () => {
            console.log(this.name);
        };

        inner();
    }
};

user.show();

// Output:
// Tarun
```

Flow:

```
user.show()
     ↓
show ka this = user
     ↓
inner arrow function
     ↓
arrow apna this nahi banata
     ↓
surrounding this leta hai
     ↓
user
```

---

6. Array me `this`

6. Array me `this`

Array khud `this` ka special case nahi hai.

Example:

JavaScript

```
const numbers = [10, 20, 30];

numbers.forEach(function (num) {
    console.log(num);
});

// Output:
// 10
// 20
// 30
```

Yaha `this` ka relation **array se automatically nahi hai**.

7. `forEach()` me `this` ⭐

`forEach()` ka optional third argument `thisArg` hota hai.

JavaScript

```
const user = {
    name: "Tarun"
};

const numbers = [10, 20, 30];

numbers.forEach(function (num) {
    console.log(this.name, num);
}, user);

// Output:
// Tarun 10
// Tarun 20
// Tarun 30
```

Yaha:

JavaScript

```
}, user);
```

`user` ko `this` ke liye provide kiya gaya.

So:

```
forEach callback
       ↓
this = user
```

8. Arrow function + `forEach()` ⚠️

JavaScript

```
const user = {
    name: "Tarun"
};

const numbers = [10, 20, 30];

numbers.forEach((num) => {
    console.log(this.name, num);
}, user);
```

Yaha `user` dene ke baad bhi arrow function ka `this` `user` nahi banega.

Kyun?

> **Arrow function `thisArg` ko ignore karta hai because arrow function ka apna `this` nahi hota.**

10. Object ke andar Array

JavaScript

```javascript
const company = {

    name: "ABC",

    employees: ["Tarun", "Rahul", "Aman"],

    showEmployees: function () {

        console.log(this.name);

        this.employees.forEach(function (employee) {
            console.log(employee);
        });
    }
};

company.showEmployees();

// Output:
// ABC
// Tarun
```

Js code

Yaha:

```javascript
this.name
```

→ `company.name`

Aur:

```javascript
this.employees
```

→ `company.employees`

---

11. Object + Array + Arrow Function ⭐

Ab arrow function ka actual advantage:

JavaScript

```
const company = {

    name: "ABC",

    employees: ["Tarun", "Rahul", "Aman"],

    showEmployees: function () {

        this.employees.forEach((employee) => {
            console.log(this.name, employee);
        });
    }
};

company.showEmployees();

// Output:
// ABC Tarun
// ABC Rahul
```

Flow:

```
company.showEmployees()
          ↓
this = company
          ↓
forEach arrow function
          ↓
arrow surrounding this leta hai
          ↓
this = company
```

Isliye:

```JavaScript
this.name
```

arrow ke andar bhi `"ABC"` milta hai.

Js code

12. `map()` + `this`

Example:

JavaScript

```
const user = {
    name: "Tarun",
    numbers: [10, 20, 30],

    double: function () {

        return this.numbers.map((num) => {
            return num * 2;
        });
    }
};

console.log(user.double());

// Output:
// [20, 40, 60]
```

Js code

Yaha `this.numbers` :

```
this → user
```

So:

```javascript
this.numbers
```

means:

```javascript
user.numbers
```

13. `this` in Array object methods

Array ke andar object methods bana sakte ho:

JavaScript

```
const users = [
    {
        name: "Tarun",

        greet() {
            console.log("Hello " + this.name);
        }
    },

    {
        name: "Rahul",

        greet() {
            console.log("Hello " + this.name);
        }
    }
];

users[0].greet();
```

Js code

Yaha:

JavaScript

```
users[0].greet()
```

to:

```
this → users[0]
```

Aur:

JavaScript

```
users[1].greet()
```

to:

```
this → users[1]
```

14. `this` ka main rule ⭐ ⭐ ⭐

Ye 4 cases yaad rakho:

Case 1 — Object method

JavaScript

```
user.show();
```

```
this → user
```

Case 2 — Normal function

JavaScript

```
show();
```

`this` call/context par depend karega.

Case 3 — Arrow function

JavaScript

```
const show = () => {};
```

```
this → surrounding lexical this
```

Case 4 — Constructor

JavaScript

```
function User(name) {
    this.name = name;
}

const u = new User("Tarun");

console.log(u.name);

// Output:
// Tarun
```

Js code

🔥 Array + Object + `this` — Quick Revision

Situation	`this`
`user.show()` normal method	`user`
Direct normal function call	Call/context par depend
Arrow function	Surrounding `this`
`forEach(function(){})`	`thisArg` diya ho to uske according
`forEach(() => {})`	Surrounding `this`
`users[0].show()`	`users[0]`
`new User()`	New object

Js code

Sabse important example:

JavaScript

```
const company = {
    name: "ABC",
    employees: ["Tarun", "Rahul"],

    show() {
        this.employees.forEach((employee) => {
            console.log(this.name, employee);
        });
    }
};

company.show();

// Output:
// ABC Tarun
// ABC Rahul
```

Yaha `show()` ka `this = company` hai, aur arrow function ne wahi surrounding `this` use kiya. यही **lexical `this`** hai.

12. `map()` ⭐⭐⭐

Array ke har element ko transform karke **new array return** karta hai.

JavaScript

```
const numbers = [1, 2, 3];

const result = numbers.map((num) => {
    return num * 2;
});

console.log(result);

// Output:
// [2, 4, 6]
```

Difference:

```
forEach → kaam karo
map     → new array banao
```

Example:

JavaScript

```
const names = ["tarun", "rahul", "aman"];

const upperNames = names.map(name => name.toUpperCase());

console.log(upperNames);

// Output:
// [ 'TARUN', 'RAHUL', 'AMAN' ]
```

13. `filter()` ⭐⭐⭐

Condition ke basis par elements select karta hai.

JavaScript

```javascript
const numbers = [10, 15, 20, 25, 30];

const result = numbers.filter(num => num > 20);

console.log(result);

// Output:
// [25, 30]
```

14. `find()`

Pehla matching element return karta hai.

JavaScript

```javascript
const numbers = [10, 20, 30, 40];

const result = numbers.find(num => num > 20);

console.log(result);

// Output:
// 30
```

`find()` sirf **first match** deta hai.

15. `findIndex()`

Pehle matching element ka index deta hai.

JavaScript

```
const numbers = [10, 20, 30, 40];

const result = numbers.findIndex(num => num > 20);

console.log(result);

// Output:
// 2
```

16. `some()`

Check karta hai ki **kam se kam ek element** condition satisfy karta hai ya nahi.

JavaScript

```
const numbers = [10, 20, 30];

console.log(numbers.some(num => num > 25));

// Output:
// true
```

17. `every()`

Check karta hai ki **saare elements** condition satisfy karte hain ya nahi.

JavaScript

```
const numbers = [10, 20, 30];

console.log(numbers.every(num => num > 5));

// Output:
// true
```

JavaScript

```
console.log(numbers.every(num => num > 15));

// Output:
// false
```

18. `reduce()` ⭐⭐⭐

Array ko ek single value me convert karne ke liye.

Example sum:

JavaScript

```javascript
const numbers = [10, 20, 30];

const total = numbers.reduce((sum, num) => {
    return sum + num;
}, 0);

console.log(total);

// Output:
// 60
```

🔥 Hoisting kya hai?

Hoisting ka matlab hai JavaScript execution se pehle declarations ko process karta hai, isliye kuch variables/functions ko code me declaration se pehle access kar sakte ho.

Lekin **sab declarations same tarike se behave nahi karte**.

---

1. Function Declaration — Fully Hoisted ⭐ ⭐ ⭐

JavaScript

```javascript
sayHello();

function sayHello() {
    console.log("Hello Tarun");
}

// Output:
// Hello Tarun
```

Ye kaam karta hai kyunki **function declaration fully hoisted hoti hai**.

Conceptually JavaScript ise aise treat kar sakta hai:

JavaScript

```
function sayHello() {
    console.log("Hello Tarun");
}

sayHello();
```

Isliye:

```
Function Declaration
        ↓
Fully Hoisted
        ↓
Declaration se pehle call kar sakte ho
```

2. `var` Hoisting

JavaScript

```
console.log(age);

var age = 22;
```

Output:

```
undefined
```

Error nahi aayega.

Conceptually:

JavaScript

```
var age;

console.log(age);

age = 22;
```

3. `let` Hoisting ⭐

Ye thoda confusing hai.

```javascript
console.log(age);

let age = 22;
```

Output:

```
ReferenceError: Cannot access 'age' before initialization
```

Logically `let` bhi creation phase me register hota hai, **lekin initialization declaration wali line par hoti hai**.

Is period ko **TDZ — Temporal Dead Zone** bolte hain.

```
let age
   ↓
Created
   ↓
TDZ
   ↓
let age = 22
   ↓
Initialized
   ↓
ab access allowed
```

4. `const` Hoisting

JavaScript

```
console.log(age);

const age = 22;
```

Output:

```
ReferenceError: Cannot access 'age' before initialization
```

`const` bhi `let` ki tarah **TDZ** me hota hai.

Declaration	Hoisted?	Before declaration access
`var`	Yes	`undefined`
`let`	Yes*	ReferenceError
`const`	Yes*	ReferenceError
Function declaration	Yes	Successfully executes

* `let` / `const` hoist hote hain, but initialization se pehle **TDZ** me rehte hain.

Js code

```javascript
sayHello();

const sayHello = function () {
    console.log("Hello");
};

// Output:
// ReferenceError
```

Arrow Function

```javascript
sayHello();

const sayHello = () => {
    console.log("Hello");
};

// Output:
// ReferenceError
```

10. Hoisting with Scope

JavaScript

```
var x = 10;

function test() {
    console.log(x);

    var x = 20;
}

test();
```

Output:

```
undefined
```

Kyun?

Function ke andar `var x` hoist ho jayega:

JavaScript

```
var x = 10;

function test() {

    var x;

    console.log(x);

    x = 20;
}
```

Isliye `undefined`.

Js code

11. Important Interview Example

JavaScript

```javascript
var x = 10;

function test() {
    console.log(x);
    var x = 20;
    console.log(x);
}

test();
```

Output:

```
undefined
20
```

Js code

Reason:

```
test()
 ↓
local x created
 ↓
x = undefined
 ↓
console.log(x) → undefined
 ↓
x = 20
 ↓
console.log(x) → 20
```

12. Function Hoisting + `var`

JavaScript

```
console.log(a);

var a = 10;

function test() {
    console.log("Hello");
}
```

Output:

```
undefined
```

`test` function declaration hoisted hai, `a` bhi hoisted hai.

Syntax + Example + Output + Purpose (kyun use karte hain) ke saath diya hai.

Upgrade Share

JavaScript String Methods

No.	Method / Property	Example	Output	Kyu use karte hain?
1	`length`	`"Tarun".length`	`5`	String me kitne characters hain, ye pata karne ke liye
2	`charAt()`	`"Tarun".charAt(2)`	`"r"`	Particular index ka character nikalne ke liye
3	`charCodeAt()`	`"ABC".charCodeAt(0)`	`65`	Character ka UTF-16 code nikalne ke liye
4	`at()`	`"Tarun".at(-1)`	`"n"`	Index se character nikalne ke liye; negative index bhi de sakte hain
5	`indexOf()`	`"hello".indexOf("l")`	`2`	Kisi character/substring ka **first index** find karne ke liye
6	`lastIndexOf()`	`"hell…stIndexOf("l")`	`3`	Kisi character/substring ka **last index** find karne ke liye

Ask anything Think

Js code

6	`lastIndexOf()`	`"hello".lastIndexOf("l")`	`3`	Kisi **index** find karne ke liye
7	`includes()`	`"hello".includes("ell")`	`true`	Check karne ke liye ki substring present hai ya nahi
8	`startsWith()`	`"JavaScript".startsWith("Java")`	`true`	Check karne ke liye string kis text se start ho rahi hai
9	`endsWith()`	`"JavaScript".endsWith("Script")`	`true`	Check karne ke liye string kis text par end ho rahi hai
10	`slice()`	`"JavaScript".slice(0,4)`	`"Java"`	String ka ek portion/extract nikalne ke liye
11	`substring()`	`"JavaScript".substring(4,10)`	`"Script"`	String ka specific portion nikalne ke liye
12	`substr()`	`"JavaScript".substr(4,6)`	`"Script"`	Starting index se specified length nikalna; **deprecated**

Js code

12	`substr()`	`"JavaScript".substr(4,6)`	`"Script"`	Sta... nikal... deprecated
13	`toUpperCase()`	`"tarun".toUpperCase()`	`"TARUN"`	String ko uppercase me convert karne ke liye
14	`toLowerCase()`	`"TARUN".toLowerCase()`	`"tarun"`	String ko lowercase me convert karne ke liye
15	`trim()`	`" Tarun ".trim()`	`"Tarun"`	Starting aur ending ke spaces remove karne ke liye
16	`trimStart()`	`" Tarun ".trimStart()`	`"Tarun "`	Sirf starting spaces remove karne ke liye
17	`trimEnd()`	`" Tarun ".trimEnd()`	`" Tarun"`	Sirf ending spaces remove karne ke liye
18	`concat()`	`"Hello".concat(" ","Tar...`	`"Hello Tarun"`	Do ya multiple strings combine karne ke liye

Js code

19	`repeat()`	`"Hi ".repeat(3)`	`"Hi Hi Hi "`	Stri karne ke liye
20	`replace()`	`"I like Java".replace("Java","JS")`	`"I like JS"`	Matching text ko replace karne ke liye
21	`replaceAll()`	`"a a a".replaceAll("a","b")`	`"b b b"`	String me **saari matching values** replace karne ke liye
22	`split()`	`"A,B,C".split(",")`	`["A","B","C"]`	String ko array me convert/break karne ke liye
23	`padStart()`	`"5".padStart(3,"0")`	`"005"`	String ke beginning me padding add karne ke liye
24	`padEnd()`	`"5".padEnd(3,"0")`	`"500"`	String ke end me padding add karne ke liye
25	`match()`	`"cat`	`["at","at"]`	Regex se matching text find

Js code

25	`match()`	`"cat bat".match(/at/g)`	`["at","at"]`	Reg karne ke liye
26	`matchAll()`	`"cat bat".matchAll(/at/g)`	Iterator	Regex ke **all matches + details** nikalne ke liye
27	`search()`	`"Hello World".search(/World/)`	`6`	Regex ka first matching index find karne ke liye
28	`localeCompare()`	`"apple".localeCompare("banana")`	Negative	Do strings ko language/locale rules ke according compare karne ke liye
29	`normalize()`	`"é".normalize()`	`"é"`	Unicode characters ko standardized form me normalize karne ke liye
30	`toString()`	`(123).toString()`	`"123"`	Value ko string representation me convert karne ke liye

Js code

JavaScript Array Methods — Complete Table

Upgrade Share ...

No.	Method	Example	Output / Result	Kyu use karte hain?
1	`length`	`[10,20,30].length`	`3`	Array me kitne elements hain
2	`at()`	`[10,20,30].at(-1)`	`30`	Index se element nikalna, negative index bhi
3	`push()`	`a.push(40)`	`[10,20,30,40]`	**End me** element add karna
4	`pop()`	`a.pop()`	Last element remove	**End se** element remove karna
5	`unshift()`	`a.unshift(5)`	`[5,10,20,30]`	**Beginning me** element add karna
6	`shift()`	`a.shift()`	First element remove	**Beginning se** element remove karna
7	`slice()`	`[10,20,...0].slice(1,3)`	`[20,30]`	Array ka portion copy/extract karna

Js code

7	`slice()`	`[10,20,30,40].slice(1,3)`	`[20,30]`	Array ka portion copy/extract karna
8	`splice()`	`a.splice(1,1)`	Index 1 ka element remove	Add/remove/replace karna
9	`concat()`	`[1,2].concat([3,4])`	`[1,2,3,4]`	Arrays combine karna
10	`indexOf()`	`[10,20,30].indexOf(20)`	`1`	Element ka first index find karna
11	`lastIndexOf()`	`[10,20,20].lastIndexOf(20)`	`2`	Element ka last index find karna
12	`includes()`	`[10,20,30].includes(20)`	`true`	Element present hai ya nahi
13	`find()`	`[5,12,...ind(x=>x>10)`	`12`	Condition satisfy karne wala **first element**

Js code

		s(20)		
13	`find()`	`[5,12,20].find(x=>x>10)`	`12`	Condition satisfy karne wala **first element**
14	`findIndex()`	`[5,12,20].findIndex(x=>x>10)`	`1`	Condition satisfy karne wale first element ka index
15	`findLast()`	`[5,12,20].findLast(x=>x>10)`	`20`	Condition satisfy karne wala **last element**
16	`findLastIndex()`	`[5,12,20].findLastIndex(x=>x>10)`	`2`	Condition satisfy karne wale last element ka index
17	`filter()`	`[5,12,20].filter(x=>x>10)`	`[12,20]`	Condition ke according multiple elements filter karna
18	`map()`	`[1,2,3].map(x=>x*2)`	`[2,4,6]`	Har element ko transform karna
19	`forEach()`	`a.forEa (x=>conso le.log(x))`	Each element print	Har element par operation karna

Js code

		)		
19	`forEach()`	`a.forEach(x=>console.log(x))`	Each element print	Har element par operation karna
20	`reduce()`	`[1,2,3].reduce((a,b)=>a+b,0)`	`6`	Array ko ek single result me convert karna
21	`reduceRight()`	`[1,2,3].reduceRight(...)`	Right → Left	Right side se reduce karna
22	`some()`	`[1,2,3].some(x=>x>2)`	`true`	**At least one** element condition satisfy karta hai?
23	`every()`	`[1,2,3].every(x=>x>0)`	`true`	**All** elements condition satisfy karte hain?
24	`sort()`	`[3,1,2].sort()`	`[1,2,3]`	Elements sort karna
25	`reverse()`	`[1,2,3].reverse()`	`[3,2,1]`	Array reverse karna
26	`toReversed()`	`[1,2,3].toReversed`	`[3,2,1]`	Reverse ki **new array** banana

Js code

26	`toReversed()`	`[1,2,3].toReversed()`	`[3,2,1]`	Reverse ki **new array** banana
27	`toSorted()`	`[3,1,2].toSorted()`	`[1,2,3]`	Sorted **new array** banana
28	`toSpliced()`	`a.toSpliced(1,1)`	New modified array	`splice()` ka non-mutating version
29	`fill()`	`[1,2,3].fill(0)`	`[0,0,0]`	Elements ko same value se fill karna
30	`copyWithin()`	`[1,2,3,4].copyWithin(1,2)`	`[1,3,4,4]`	Array ke andar values copy karna
31	`join()`	`[1,2,3].join("-")`	`"1-2-3"`	Array ko string me convert karna
32	`flat()`	`[1,[2,[3]]].fl...,2)`	`[1,2,3]`	Nested arrays ko flatten karna

Js code

31	`join()`	`[1,2,3].join("-")`	`"1-2-3"`	Array ko string me convert karna
32	`flat()`	`[1,[2,[3]]].flat(2)`	`[1,2,3]`	Nested arrays ko flatten karna
33	`flatMap()`	`[1,2].flatMap(x=>[x,x*2])`	`[1,2,2,4]`	`map()` + `flat()` ek saath
34	`entries()`	`a.entries()`	Iterator	Index + value pairs nikalna
35	`keys()`	`a.keys()`	Iterator	Array ke indexes nikalna
36	`values()`	`a.values()`	Iterator	Array ke values nikalna

JavaScript Math Methods

S.No.	Method	Kya karta hai	Example	Output
1	`Math.round()`	Nearest integer	`Math.round(4.6)`	`5`
2	`Math.floor()`	Neeche round karta hai	`Math.floor(4.9)`	`4`
3	`Math.ceil()`	Upar round karta hai	`Math.ceil(4.1)`	`5`
4	`Math.trunc()`	Decimal hata deta hai	`Math.trunc(4.9)`	`4`
5	`Math.abs()`	Negative ko positive karta hai	`Math.abs(-10)`	`10`
6	`Math.max()`	Maximum number deta hai	`Math.max(10,20,5)`	`20`
7	`Math.min()`	Minimum number deta hai	`Math.min(10,20,5)`	`5`

8	`Math.pow()`	Power nikalta hai	`Math.pow(2,3)`	`8`
9	`Math.sqrt()`	Square root nikalta hai	`Math.sqrt(25)`	`5`
10	`Math.cbrt()`	Cube root nikalta hai	`Math.cbrt(27)`	`3`
11	`Math.random()`	Random number deta hai	`Math.random()`	`0 to <1`
12	`Math.sign()`	Number ka sign batata hai	`Math.sign(-10)`	`-1`
13	`Math.log()`	Natural logarithm	`Math.log(Math.E)`	`1`
14	`Math.log10()`	Base-10 logarithm	`Math.log10(100)`	`2`
15	`Math.exp()`	`e` ki power nikalta hai	`Math.exp(1)`	`2.718...`

Js code

b562cd0

15	`Math.exp()`	`e` ki power nikalta hai	`Math.exp(1)`	`2.718...`
16	`Math.sin()`	Sine nikalta hai	`Math.sin(0)`	`0`
17	`Math.cos()`	Cosine nikalta hai	`Math.cos(0)`	`1`
18	`Math.tan()`	Tangent nikalta hai	`Math.tan(0)`	`0`

Math Constants

S.No.	Constant	Kya karta hai	Example	Output
1	`Math.PI`	π ki value	`Math.PI`	`3.14159...`
2	`Math.E`	Euler number	`Math.E`	`2.71828...`
3	`Math.SQRT2`	√2 ki value	`Math.SQRT2`	`1.41421...`

Js code

```
console.log(Math.round(4.6));        // 5
console.log(Math.floor(4.9));       // 4
console.log(Math.ceil(4.1));        // 5
console.log(Math.trunc(4.9));       // 4
console.log(Math.abs(-10));         // 10
console.log(Math.max(10, 20, 5));   // 20
console.log(Math.min(10, 20, 5));   // 5
console.log(Math.pow(2, 3));        // 8
console.log(Math.sqrt(25));         // 5
console.log(Math.cbrt(27));         // 3
console.log(Math.sign(-10));        // -1
console.log(Math.log10(100));       // 2
console.log(Math.sin(0));           // 0
console.log(Math.cos(0));           // 1
console.log(Math.tan(0));           // 0

console.log(Math.PI);               // 3.141592653589793
console.log(Math.E);                // 2.718281828459045

console.log(Math.random());         // 0 to <1 random number
```

gg

Js code

Type	Example
Function Declaration	`function add(){}`
Function Expression	`const add = function(){}`
Arrow Function	`const add = () => {}`
Anonymous Function	`function(){}`
IIFE	`(function(){})()`
Callback Function	`arr.forEach(function(){})`
Recursive Function	Function calling itself
Nested Function	Function inside function
Method	Object ke andar function
Async Function	`async function getData(){}`
Generator Function	`function* gen(){}`

2. Function with Parameter

Parameter function ko input dene ke liye hota hai.

JavaScript

```javascript
function greet(name) {
    console.log("Hello", name);
}

greet("Tarun");

// Output:
// Hello Tarun
```

Yaha:

```
name → parameter
"Tarun" → argument
```

3. Multiple Parameters

JavaScript

```javascript
function add(a, b) {
    console.log(a + b);
}

add(10, 20);

// Output:
// 30
```

```
a = 10
b = 20
```

4. `return` ⭐⭐⭐

`return` function se value bahar bhejta hai.

JavaScript

```javascript
function add(a, b) {
    return a + b;
}

const result = add(10, 20);

console.log(result);

// Output:
// 30
```

Difference:

JavaScript

```
console.log()
```

→ value print karta hai.

JavaScript

```
return
```

→ value function se bahar deta hai.

---

5. Function Expression

Function ko variable me store kar sakte ho.

JavaScript

```
const greet = function () {
    console.log("Hello");
};

greet();

// Output:
// Hello
```

Js code

Parameter:

JavaScript

```
const greet = (name) => {
    console.log("Hello", name);
};

greet("Tarun");

// Output:
// Hello Tarun
```

7. Arrow Function Short Form

Agar sirf ek expression return karna hai:

JavaScript

```
const add = (a, b) => a + b;

console.log(add(10, 20));

// Output:
// 30
```

Ye:

JavaScript

```
const add = (a, b) => a + b;
```

same hai:

Ye:

JavaScript

```
const add = (a, b) => a + b;
```

same hai:

JavaScript

```
const add = (a, b) => {
    return a + b;
};
```

8. Default Parameter

Agar argument nahi diya to default value use hogi.

JavaScript

```
function greet(name = "Guest") {
    console.log("Hello", name);
}

greet();

// Output:
// Hello Guest
```

JavaScript

```
greet("Tarun");

// Output:
// Hello Tarun
```

9. Rest Parameter `...`

Multiple arguments ko array me collect karta hai.

JavaScript

```
function add(...numbers) {
    console.log(numbers);
}

add(10, 20, 30, 40);

// Output:
// [10, 20, 30, 40]
```

Calculation:

Calculation:

JavaScript

```
function add(...numbers) {
    return numbers.reduce((sum, num) => sum + num, 0);
}

console.log(add(10, 20, 30));

// Output:
// 60
```

10. Function as Argument ⭐ ⭐ ⭐

JavaScript me function ko doosre function me argument ki tarah pass kar sakte ho.

JavaScript

```javascript
function greet(name) {
    console.log("Hello", name);
}

function processUser(callback) {
    callback("Tarun");
}

processUser(greet);

// Output:
// Hello Tarun
```

Yaha `greet` ek **callback function** hai.

11. Anonymous Function

Jis function ka naam nahi hota:

JavaScript

```javascript
const greet = function () {
    console.log("Hello");
};

greet();

// Output:
// Hello
```

Function ka naam nahi hai, lekin `greet` variable us function ko hold kar raha hai.

12. IIFE ⭐

Immediately Invoked Function Expression

Function banate hi execute ho jata hai.

JavaScript

```
(function () {
    console.log("Hello");
})();

// Output:
// Hello
```

Arrow IIFE:

JavaScript

```
(() => {
    console.log("Hello");
})();

// Output:
// Hello
```

13. Nested Function

Function ke andar function.

JavaScript

```
function outer() {

    function inner() {
        console.log("Inner");
    }

    inner();
}

outer();

// Output:
// Inner
```

Structure:

```
outer()
   ↓
inner()
```

14. Recursive Function ⭐⭐

Function khud ko call kare.

Example factorial:

JavaScript

```javascript
function factorial(n) {

    if (n === 1) {
        return 1;
    }

    return n * factorial(n - 1);
}

console.log(factorial(5));

// Output:
// 120
```

Flow:

```
factorial(5)
 ↓
5 × factorial(4)
 ↓
5 × 4 × factorial(3)
 ↓
5 × 4 × 3 × 2 × 1
 ↓
120
```

16. Object ke andar Function → Method

Object ke andar function ko **method** bolte hain.

JavaScript

```javascript
const user = {
    name: "Tarun",

    greet: function () {
        console.log("Hello", this.name);
    }
};

user.greet();

// Output:
// Hello Tarun
```

Modern shorthand:

JavaScript

```
const user = {
    name: "Tarun",

    greet() {
        console.log("Hello", this.name);
    }
};

user.greet();

// Output:
// Hello Tarun
```

17. `this` with Function ⭐ ⭐ ⭐

Object method me `this` current object ko refer karta hai.

JavaScript

```
const user = {
    name: "Tarun",

    show() {
        console.log(this.name);
    }
};

user.show();

// Output:
// Tarun
```

1. Scope kya hai?

Scope = kisi variable ko code ke kis part me access kar sakte ho.

Example:

JavaScript

```
let name = "Tarun";

console.log(name);

// Output:
// Tarun
```

`name` yaha accessible hai.

3. Global Scope

Jo variable kisi function/block ke bahar declare ho.

JavaScript

```javascript
let name = "Tarun";

function greet() {
    console.log(name);
}

greet();

// Output:
// Tarun
```

`name` global scope me hai, isliye function ke andar access ho gaya.

4. Function Scope

Function ke andar declared variable function ke andar accessible hota hai.

JavaScript

```javascript
function test() {

    let age = 22;

    console.log(age);
}

test();

// Output:
// 22
```

6. Ab aata hai Lexical Scope ⭐ ⭐ ⭐

Lexical scope ka matlab hai variable ka scope uske code me likhe/defined position se decide hota hai.

Simple language:

> **Function ke andar ka code apne outer scope ke variables ko access kar sakta hai.**

Example:

Js code

```
let name = "Tarun";

function outer() {

    let age = 22;

    function inner() {
        console.log(name);
        console.log(age);
    }

    inner();
}

outer();

// Output:
// Tarun
// 22
```

`inner()` ke paas:

```
inner scope
    ↓
outer scope
    ↓
global scope
```

Isliye `inner()`:

- apne variables
- outer function ke variables
- global variables

access kar sakta hai.

Js code

7. Lexical Scope ka diagram ⭐ ⭐ ⭐

JavaScript

```
let a = 10;

function outer() {

    let b = 20;

    function inner() {

        let c = 30;

        console.log(a);
        console.log(b);
        console.log(c);
    }

    inner();
}
```

Js code

Scope chain:

```
GLOBAL SCOPE
│
└── a = 10
     │
     ↓
OUTER SCOPE
│
└── b = 20
     │
     ↓
INNER SCOPE
│
└── c = 30
```

`inner()` jab `a` dhundhega:

Js code

`inner()` jab `a` dhundhega:

```
inner
 ↓
outer
 ↓
global
```

Mil gaya → `10`.

`b`:

```
inner
 ↓
outer
 ↓
20
```

`c`:

```
inner
 ↓
30
```

Js code

9. Bahut important: Outer function inner variable access nahi kar sakta

Ye difference samjho.

JavaScript

```
function outer() {

    let outerValue = 10;

    function inner() {

        let innerValue = 20;

        console.log(outerValue);
        console.log(innerValue);
    }

    inner();

    console.log(outerValue);
    console.log(innerValue);
}
```

Js code

JavaScript

```
    console.log(outerValue);
    console.log(innerValue);
}

outer();
```

Output:

```
10
20
10

ReferenceError: innerValue is not defined
```

Kyun?

Scope chain **andar ki taraf reverse nahi jaati.**

```
outer
  ↓
inner
```

Inner outer ko access kar sakta hai:

```
inner → outer ✅
```

Lekin outer inner ko access nahi kar sakta:

```
outer → inner ❌
```

10. Lexical Scope vs Dynamic Scope

JavaScript **lexical scoped** language hai.

Example:

JavaScript

```
let name = "Global";

function outer() {

    let name = "Outer";

    function inner() {
        console.log(name);
    }

    inner();
}

outer();

// Output:
// Outer
```

11. Function ko kahaan call kiya, usse lexical scope change nahi hota

Ye bahut important hai.

JavaScript

```javascript
let name = "Global";

function inner() {
    console.log(name);
}

function outer() {

    let name = "Outer";

    inner();
}

outer();

// Output:
// Global
```

Js code

12. Lexical Scope ka easiest example

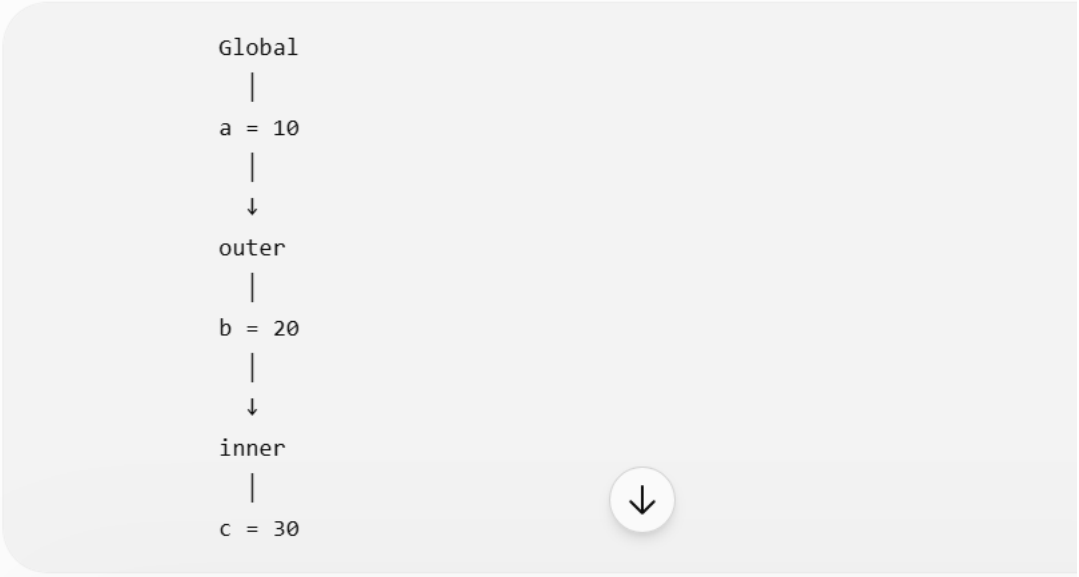
JavaScript

```
const a = 10;

function outer() {

    const b = 20;

    function inner() {

        const c = 30;

        console.log(a);
        console.log(b);
        console.log(c);
    }

    inner();
}

outer();
```

Js code

```
// Output:
// 10
// 20
// 30
```

Yaha `inner()` apne surrounding lexical scopes ko access kar raha hai.



🔥 Closure kya hai?

Simple definition:

> **Jab ek inner function apne outer function ke variables ko remember karta hai, even outer function execute ho chuka ho, usse Closure kehte hain.**

Sabse simple example:

Js code

```javascript
function outer() {

    let count = 0;

    function inner() {
        count++;
        console.log(count);
    }

    return inner;
}

const counter = outer();

counter();
// Output: 1

counter();
// Output: 2

counter();
// Output: 3
```

Js code

1. Normally kya hona chahiye?

`outer()` ke andar:

JavaScript

```
let count = 0;
```

`outer()` execute hua:

JavaScript

```
const counter = outer();
```

Normally tum sochoge ki `outer()` khatam hone ke baad:

```
outer()
   ↓
count = 0
   ↓
outer finish
   ↓
count destroy?
```

Lekin `inner()` ne `count` ko **remember** kar liya.

Isliye:

JavaScript

```
counter();
// 1

counter();
// 2

counter();
// 3
```

`inner()` ke paas:

```
inner()
  ↓
count
  ↓
outer ka variable
```

Closure ki wajah se `inner` apne surrounding lexical environment ko access kar sakta hai.

---

4. Closure ka real-life example

Socho ek **locker** hai.

```
outer()
   ↓
locker create
   ↓
count = 0
   ↓
inner function ko locker ki key mil gayi
```

`outer()` finish ho gaya, lekin `inner()` ke paas locker ki access hai.

```
inner()
   ↓
count access
   ↓
change count
```

Ye closure ka basic idea hai.

5. Closure ka ek aur simple example

JavaScript

```
function greet(name) {

    return function () {
        console.log("Hello", name);
    };
}

const greetTarun = greet("Tarun");

greetTarun();

// Output:
// Hello Tarun
```

Yaha:

JavaScript

```
greet("Tarun")
```

Yaha:

```javascript
greet("Tarun")
```

`name = "Tarun"` create hua.

`greet()` finish ho gaya.

Lekin returned function ne `name` ko remember kar liya.

```
greet()
 |
 └── name = "Tarun"
       ↑
       |
returned function
       |
       ↓
greetTarun()
```

Js code

f-79dd3b562cd0

6. Multiple closures ⭐

JavaScript

```
function counter() {

    let count = 0;

    return function () {
        count++;
        return count;
    };
}

const counter1 = counter();
const counter2 = counter();

console.log(counter1());
// Output: 1

console.log(counter1());
// Output: 2

console.log(counter2());
// Output: 1
```

Js code

JavaScript

```
console.log(counter2());
// Output: 1

console.log(counter1());
// Output: 3
```

Important:

`counter1` aur `counter2` ke **alag-alag closures** hain.

```
counter1
   ↓
count = 0 → 1 → 2 → 3



counter2
   ↓
count = 0 → 1
```

Isliye `counter2()` ka count `1` se start hua.

7. Closure se data private karna ⭐⭐

Ye closure ka important real-world use hai.

JavaScript

```javascript
function bankAccount() {

    let balance = 1000;

    return {
        deposit(amount) {
            balance += amount;
        },

        getBalance() {
            return balance;
        }
    };
}

const account = bankAccount();

console.log(account.getBalance());
// Output: 1000
```

Js code

```
console.log(account.getBalance());
// Output: 1000

account.deposit(500);

console.log(account.getBalance());
// Output: 1500
```

Bahaar se:

```
JavaScript

console.log(account.balance);
```

Output:

```
undefined
```

`balance` directly accessible nahi hai.

Lekin methods:

```
deposit()
   ↓
balance access

getBalance()
   ↓
balance access
```

Ye **data privacy / encapsulation** ka simple example hai.

Js code

```javascript
function outer() {

    let x = 10;

    return function inner() {
        console.log(x);
    };
}

const fn = outer();

fn();

// Output:
// 10
```

Ab `outer()` finish hone ke **baad bhi** `inner()` `x` ko access kar raha hai.

Ye closure hai.

```
Lexical Scope
      ↓
Inner function outer variable ko access karta hai
      ↓
Outer function finish
      ↓
Inner function variable ko remember karta hai
      ↓
CLOSURE
```

Js code

d3b562cd0

> **Arrow function apna khud ka `this` nahi banata. Wo apne surrounding (lexical) scope se `this` leta hai.**

Isko examples se samjho.

---

1. Normal function ka `this`

JavaScript

```
const user = {
    name: "Tarun",

    show: function () {
        console.log(this.name);
    }
};

user.show();

// Output:
// Tarun
```

Yaha normal function ka `this` **jis object ke through function call hua**, usko refer karta hai.

```
user.show()
    ↓
this
    ↓
user
```

Isliye:

JavaScript

```
this.name
// Tarun
```

2. Arrow function ka `this`

Ab:

JavaScript

```
const user = {
    name: "Tarun",

    show: () => {
        console.log(this.name);
    }
};

user.show();
```

Output generally:

```
undefined
```

Kyun?

Arrow function ne `this` **user object se nahi liya.**

Js code

3. Surrounding `this` ka matlab kya hai?

Ye example dekho:

JavaScript

```
const user = {
    name: "Tarun",

    show: function () {

        const inner = () => {
            console.log(this.name);
        };

        inner();
    }
};

user.show();

// Output:
// Tarun
```

Js code

4. Normal function vs Arrow function

JavaScript

```
const user = {
    name: "Tarun",

    show: function () {

        console.log(this.name);

        const arrow = () => {
            console.log(this.name);
        };

        function normal() {
            console.log(this.name);
        }

        arrow();
        normal();
    }
};

user.show();
```

Js code

`show()` ke andar:

```
show()
 ↓
this = user
```

Arrow:

```
arrow()
 ↓
this = surrounding this
 ↓
user
```

Normal function:

```
normal()
 ↓
apna this
 ↓
call ke according decide
```

Browser/strict-mode context ke according normal function ka `this` alag ho sakta hai.

5. Arrow function ka `this` change nahi hota

JavaScript

```
const user = {
    name: "Tarun",

    show: function () {

        const arrow = () => {
            console.log(this.name);
        };

        arrow();
    }
};

user.show();

// Output:
// Tarun
```

Arrow ka `this`:

```
arrow
 ↓
surrounding show()
 ↓
this = user
```

6. Real-world example: `setTimeout()` ⭐

Ye arrow function ka sabse useful example hai.

JavaScript

```
const user = {
    name: "Tarun",

    show: function () {

        setTimeout(() => {
            console.log(this.name);
        }, 1000);
    }
};

user.show();

// Output after 1 second:
// Tarun
```

apna `this` nahi banata.

Wo `show()` ka `this` use karta hai:

```
user.show()
     ↓
this = user
     ↓
setTimeout
     ↓
arrow function
     ↓
this = user
     ↓
Tarun
```

7. Agar `setTimeout` me normal function hota?

JavaScript

```
const user = {
    name: "Tarun",

    show: function () {

        setTimeout(function () {
            console.log(this.name);
        }, 1000);
    }
};

user.show();
```

Yaha normal function ka `this` **surrounding** `show()` **ka** `this` **automatically nahi leta.**

Isliye output environment/mode ke according `undefined` ho sakta hai.

Isi problem ko arrow function easily solve karta hai.

Higher-Order Function kya hai?

> **Jo function kisi doosre function ko argument ke roop me leta hai ya function ko return karta hai, use Higher-Order Function kehte hain.**

Do cases hain:

```
1. Function ko argument me lena
2. Function ko return karna
```

1. Function ko argument me lena ⭐ ⭐ ⭐

JavaScript

```
function greet(name) {
    console.log("Hello", name);
}

function processUser(callback) {
    callback("Tarun");
}

processUser(greet);

// Output:
// Hello Tarun
```

Yaha:

JavaScript

```
processUser(greet);
```

`processUser()` ne `greet` function ko **argument** ke roop me liya.

2. Function ko return karna

JavaScript

```
function outer() {

    function inner() {
        console.log("Hello");
    }

    return inner;
}

const result = outer();

result();

// Output:
// Hello
```

Yaha:

Yaha:

JavaScript

```
outer()
```

ek function return kar raha hai.

Isliye `outer` ek **Higher-Order Function** hai.

Js code

```
function calculate(a, b, operation) {
    return operation(a, b);
}

function add(x, y) {
    return x + y;
}

console.log(calculate(10, 20, add));

// Output:
// 30
```

Flow:

```
calculate(10, 20, add)
            ↓
        add function
            ↓
        add(10, 20)
            ↓
            30
```

Js code

`calculate()` → Higher-Order Function

`add()` → Callback Function

---

4. JavaScript ke built-in HOF ⭐ ⭐ ⭐

JavaScript me arrays ke bahut methods Higher-Order Functions hain.

`forEach()`

JavaScript

```
const numbers = [1, 2, 3];

numbers.forEach(function(num) {
    console.log(num);
});

// Output:
// 1
// 2
// 3
```

Js code

5. `map()` ⭐⭐⭐

JavaScript

```
const numbers = [1, 2, 3];

const result = numbers.map(function(num) {
    return num * 2;
});

console.log(result);

// Output:
// [2, 4, 6]
```

`map()` function ko argument me leta hai.

Therefore:

```
map() → Higher-Order Function
function(num) → Callback
```

Js code

HOF	Main use
`forEach()`	Har element par kaam
`map()`	New transformed array
`filter()`	Matching elements
`find()`	First matching element
`findIndex()`	First matching index
`some()`	At least one match
`every()`	All elements match
`reduce()`	Array ko single value me convert

Js code

S.No.	Use	Example	Output
1	Array copy	`const b = [...arr]`	`[10, 20, 30]`
2	Array merge	`const c = [...arr1, ...arr2]`	`[10, 20, 30, 40]`
3	Array me add	`const b = [...arr, 40]`	`[10, 20, 30, 40]`
4	Function arguments	`Math.max(...arr)`	`30`
5	Object copy	`const b = {...obj}`	`{ name: "Tarun", age: 22 }`
6	Object merge	`{...obj1, ...obj2}`	`{ name: "Tarun", age: 22 }`
7	Object update	`{ ...user, age: 23}`	`{ name: "Tarun", age: 23 }`

> Spread Operator (`...`) JavaScript me kisi iterable ke elements ya object ki properties ko expand karke unhe kisi naye array, object, ya function ke arguments ke roop me use karne deta hai.

Aur simple language me:

> Spread (`...`) ka matlab hai kisi collection ke andar ke elements/properties ko kholkar individual values ki tarah use karna.

Example:

JavaScript

```javascript
const arr = [10, 20, 30];

console.log(...arr);

// Output:
// 10 20 30
```

2. Array copy karna ⭐ ⭐

JavaScript

```
const arr1 = [10, 20, 30];

const arr2 = [...arr1];

console.log(arr2);

// Output:
// [10, 20, 30]
```

Ab `arr2` ek **new array** hai.

JavaScript

```
arr2.push(40);

console.log(arr1);
// Output: [10, 20, 30]

console.log(arr2);
// Output: [10, 20, 30, 40]
```

3. Do arrays merge karna ⭐

JavaScript

```
const arr1 = [10, 20];
const arr2 = [30, 40];

const result = [...arr1, ...arr2];

console.log(result);

// Output:
// [10, 20, 30, 40]
```

Without spread:

JavaScript

```
const result = [arr1, arr2];

console.log(result);

// Output:
// [[10, 20], [30, 40]]
```

4. Array me extra values add karna

JavaScript

```javascript
const arr = [20, 30];

const result = [10, ...arr, 40];

console.log(result);

// Output:
// [10, 20, 30, 40]
```

5. Function arguments me spread ⭐⭐

JavaScript

```javascript
const numbers = [10, 20, 30];

console.log(Math.max(...numbers));

// Output:
```

Actually:

```javascript
Math.max(...numbers)
```

JavaScript ise conceptually:

```javascript
Math.max(10, 20, 30)
```

ki tarah pass karta hai.

6. Function me array pass karna

```javascript
function add(a, b, c) {
    return a + b + c;
}

const numbers = [10, 20, 30];

console.log(add(...numbers));

// Output:
// 60
```

7. Object ko spread karna ⭐ ⭐

JavaScript

```
const user = {
    name: "Tarun",
    age: 22
};

const copy = {
    ...user
};

console.log(copy);

// Output:
// { name: 'Tarun', age: 22 }
```

Js code

8. Objects merge karna ⭐ ⭐ ⭐

JavaScript

```javascript
const user = {
    name: "Tarun"
};

const details = {
    age: 22,
    city: "Indore"
};

const result = {
    ...user,
    ...details
};

console.log(result);

// Output:
// { name: 'Tarun', age: 22, city: 'Indore' }
```

9. Object me new property add karna

JavaScript

```
const user = {
    name: "Tarun",
    age: 22
};

const updatedUser = {
    ...user,
    city: "Indore"
};

console.log(updatedUser);

// Output:
// { name: 'Tarun', age: 22, city: 'Indore' }
```

Original object change nahi hua:

11. String ko spread karna

JavaScript

```
const name = "Tarun";

console.log([...name]);

// Output:
// ['T', 'a', 'r', 'u', 'n']
```

String iterable hoti hai.

2. Kya Spread `for loop` jaisa hai?

Conceptually thoda related hai, but same nahi hai. ❌

Spread ko **for loop mat samjho**.

`for...of` :

JavaScript

```
const arr = [10, 20, 30];

for (const value of arr) {
    console.log(value);
}

// Output:
// 10
// 20
// 30
```

Ye **iterate** kar raha hai.

Spread:

JavaScript

```
console.log(...arr);

// Output:
// 10 20 30
```

Ye **elements ko expand** kar raha hai.

Difference:

```
for...of
   ↓
ek-ek value ko iterate karta hai
   ↓
10
20
30



spread
   ↓
values ko expand karta hai
   ↓
10 20 30
```

7. Spread aur normal array me difference

JavaScript

```
const arr = [10, 20, 30];

const a = [arr];

console.log(a);

// Output:
// [[10, 20, 30]]
```

Yaha array ke andar **poora array** rakha gaya.

Lekin:

JavaScript

```
const b = [...arr];

console.log(b);

// Output:
```

Js code

Yaha elements expand hue.

```
[arr]
 ↓
[
   [10,20,30]
]


[...arr]
 ↓
[
   10,
   20,
   30
]
```

Yaha tum **har value ke saath kuch operation** kar sakte ho:

JavaScript

```
for (const value of arr) {
    console.log("Hello", value);
}

// Output:
// Hello A
// Hello B
// Hello C
```

Spread

JavaScript

```
console.log(...arr);

// Output:
// A B C
```

Spread ke saath tum per-element processing nahi kar rahe.

Js code

9. Spread + `map()` ⭐

Agar values ko change karna hai:

JavaScript

```
const arr = [10, 20, 30];

const result = arr.map(x => x * 2);

console.log(result);

// Output:
// [20, 40, 60]
```

Yaha:

```
map → iterate + transform
spread → expand
```

Dono alag kaam hain.

1. Rest kya hai?

> **Rest Operator (`...`) multiple values ko collect karke ek single array ya object me store karta hai.**

Simple:

```
Spread → Expand / फैलाना
Rest   → Collect / इकट्ठा करना
```

Js code

6. Array Destructuring me Rest ⭐

JavaScript

```javascript
const numbers = [10, 20, 30, 40, 50];

const [first, second, ...rest] = numbers;

console.log(first);
// Output: 10

console.log(second);
// Output: 20

console.log(rest);
// Output: [30, 40, 50]
```

Yaha:

```
first  → 10
second → 20
rest   → [30, 40, 50]
```

7. Object Destructuring me Rest

JavaScript

```
const user = {
    name: "Tarun",
    age: 22,
    city: "Indore"
};

const { name, ...rest } = user;

console.log(name);
// Output: Tarun

console.log(rest);
// Output: { age: 22, city: "Indore" }
```

Yaha `rest` ne **remaining properties** collect kar li.

```
name → Tarun

rest → {
    age: 22,
```

Js code

	Spread	Rest
Symbol	`...`	`...`
Meaning	Expand	Collect
Main use	Array/Object copy, merge, function arguments	Function parameters, destructuring
Direction	Collection → individual values	Individual values → collection
Example	`[...arr]`	`function f(...args)`

S.No.	API	Kya karta hai?	Simple Example	Category
1	`process.nextTick()`	Current operation complete hote hi callback ko **nextTick queue** me execute karne ke liye rakhta hai	`process.nextTick(() => console.log("Hi"))`	Next Tick
2	`queueMicrotask()`	Callback ko **microtask queue** me daalta hai; current synchronous code ke baad execute hota hai	`queueMicrotask(() => console.log("Hi"))`	Microtask
3	`Promise.then()`	Promise complete hone ke baad callback ko **microtask queue** me daalta hai	`Promise.resolve().then(() => console.log("Done"))`	Microtask
4	`setTimeout()`	Callback ko given minimum delay ke baad **timer phase** me e te karne ke liye schedule karta hai	`setTimeout(() => console.log("Hi"), 1000)`	Timer

Js code

Share

5	`setInterval()`	Callback ko **baar-baar fixed interval** par execute karta hai	`setInterval(() => console.log("Hi"), 1000)`	Timer
6	`setImmediate()`	Callback ko Node.js event loop ke **check phase** me execute karne ke liye schedule karta hai	`setImmediate(() => console.log("Hi"))`	Check Phase
7	`fs.readFile()`	File ko asynchronously read karta hai; read complete hone par callback execute hota hai	`fs.readFile("a.txt", callback)`	I/O
8	`fs.writeFile()`	File me asynchronously data write karta hai	`fs.writeFile("a.txt", "Hello", callback)`	I/O
9	`fs.appendFile()`	Existing file ke end me data add karta hai	`fs.appendFile("a.txt", "Hello", callback)`	I/O
10	`fs.unlink()`	File delete karta hai asynchronously	`fs.unlink("a.txt", callback)`	I/O
11	`server.on("request")`	HTTP request aane par callback execute karta hai	`server.on("request", callback)`	Network
12	`socket.on("data")`	Socket se data receive hone par callback execute karta hai	`socket.on("data", callback)`	Network I/O
13	`socket.on("end")`	Socket connection close/end hone par callback execute karta hai	`socket.on("end", callback)`	Network I/O

Js code

```
// ==========================================
// Node.js Event Loop - Important APIs
// ==========================================


// 1. Synchronous code
console.log("1. Start");


// 2. process.nextTick()
// Node.js nextTick queue
process.nextTick(() => {
    console.log("2. process.nextTick");
});


// 3. queueMicrotask()
// Microtask queue
queueMicrotask(() => {
    console.log("3. queueMicrotask");
});
```

Js code

```
// 4. Promise.then()
// Promise microtask
Promise.resolve().then(() => {
    console.log("4. Promise.then");
});


// 5. setTimeout()
// Timer phase
setTimeout(() => {
    console.log("5. setTimeout");
}, 0);


// 6. setImmediate()
// Check phase
setImmediate(() => {
    console.log("6. setImmediate");
});
```

Js code

```
// 7. setInterval()
// Repeatedly execute hota hai
const interval = setInterval(() => {
    console.log("7. setInterval");

    clearInterval(interval);
}, 100);


// 8. I/O callback
const fs = require("fs");

fs.readFile(__filename, () => {
    console.log("8. fs.readFile I/O");
});
```

```
// 9. Socket / Network I/O
const net = require("net");

const server = net.createServer((socket) => {

    console.log("9. Socket connection");

    socket.on("data", (data) => {
        console.log("10. Socket data");
    });

    socket.on("end", () => {
        console.log("11. Socket end");
    });
});

server.listen(3000, () => {
    console.log("12. Server listening");
});
```

Js code

JavaScript Error Types

Share

S.No.	Error Type	Kya hota hai?	Example	Output / Error
1	`Error`	General/custom error	`throw new Error("Something went wrong")`	`Error: Something went wrong`
2	`SyntaxError`	Code ka syntax galat ho	`if (true {}`	`SyntaxError`
3	`ReferenceError`	Undefined variable ko access karo	`console.log(x)`	`ReferenceError: x is not defined`
4	`TypeError`	Wrong type ka operation	`null.name`	`TypeError`
5	`RangeError`	Value valid range ke bahar ho	`new Array(-1)`	`RangeError`
6	`URIError`	Invalid URI operation	`decodeURIComponent("%")`	`URIError`
7	`EvalError`	`eval()` se related legacy	`throw new`	`EvalError`

Js code

JavaScript

```
// 1. Error
try {
    throw new Error("Something went wrong");
} catch (err) {
    console.log(err.name);
    // Output: Error

    console.log(err.message);
    // Output: Something went wrong
}


// 2. SyntaxError
try {
    JSON.parse("{name: Tarun}");
} catch (err) {
    console.log(err.name);
    // Output: SyntaxError
}
```

Js code

JavaScript

```
// 3. ReferenceError
try {
    console.log(unknownVariable);
} catch (err) {
    console.log(err.name);
    // Output: ReferenceError
}


// 4. TypeError
try {
    const user = null;
    console.log(user.name);
} catch (err) {
    console.log(err.name);
    // Output: TypeError
}
```

```
// 5. RangeError
try {
    new Array(-1);
} catch (err) {
    console.log(err.name);
    // Output: RangeError
}


// 6. URIError
try {
    decodeURIComponent("%");
} catch (err) {
    console.log(err.name);
    // Output: URIError
}
```

Js code

```
// 7. EvalError
try {
    throw new EvalError("Eval error");
} catch (err) {
    console.log(err.name);
    // Output: EvalError
}


// 8. AggregateError
try {
    const error1 = new Error("Error 1");
    const error2 = new Error("Error 2");

    throw new AggregateError(
        [error1, error2],
        "Multiple errors"
    );

} catch (err) {
    console.log(err.name);
    // Output: AggregateError
```

Js code

```
// 9. throw err → error ko dobara throw karna
try {

    throw new Error("Original error");

} catch (err) {

    console.log(err.message);
    // Output: Original error

    // throw err;
    // Existing error ko dobara throw karega
}
```

```
// 10. finally
try {

    console.log("Try");

} catch (err) {

    console.log("Catch");

} finally {

    console.log("Finally");
}

// Output:
// Try
// Finally
```

```
Error          → General error
SyntaxError    → Syntax galat
ReferenceError → Variable nahi mila
TypeError      → Wrong type operation
RangeError     → Value range se bahar
URIError       → Invalid URI
EvalError      → Legacy eval error
AggregateError → Multiple errors

throw          → Error throw karna
catch(err)     → Error handle karna
throw err      → Error dobara throw karna
finally        → Hamesha execute hone wala cleanup block
```

JavaScript mein Memory Leak kya hota hai?

Memory Leak tab hota hai jab program ko kisi object/data ki **ab zarurat nahi hai**, lekin uska reference abhi bhi kahin se bana hua hai. Isliye **Garbage Collector us memory ko free nahi kar pata**.

Simple:

```
Object ki zarurat nahi
        ↓
Reference abhi bhi present
        ↓
Garbage Collector memory free nahi kar sakta
        ↓
Memory Leak
```

1. Unnecessary Global Variable

JavaScript

```
function test() {
    data = new Array(1000000);
}

test();
```

Agar `data` accidentally global ban gaya, to uska reference global scope mein reh sakta hai.

Better:

JavaScript

```
function test() {
    const data = new Array(1000000);
}
```

2. Event Listener remove na karna

Ye common case hai:

JavaScript

```
const button = document.querySelector("#btn");

function handleClick() {
    console.log("Clicked");
}

button.addEventListener("click", handleClick);
```

Agar component/element ko remove kar diya but listener/reference properly clean nahi kiya, long-running application mein unnecessary references memory hold kar sakte hain.

Cleanup:

Cleanup:

JavaScript

```
button.removeEventListener("click", handleClick);
```

Important: `removeEventListener()` ke liye same function reference chahiye.

---

3. `setInterval()` clear na karna

JavaScript

```
const id = setInterval(() => {
    console.log("Running...");
}, 1000);
```

Agar interval ki zarurat khatam ho gayi aur clear nahi kiya:

Agar interval ki zarurat khatam ho gayi aur clear nahi kiya:

JavaScript

```
clearInterval(id);
```

to callback continuously run karta rahega aur unnecessary resources/references hold kar sakta hai.

---

4. `setTimeout()` / async callbacks

JavaScript

```
setTimeout(() => {
    console.log("Hello");
}, 60000);
```

Agar callback closure ke through large data ko reference kar raha hai, timer complete hone tak wo data retain ho sakta hai.

---

5. Closure ki wajah se

JavaScript

```
function outer() {
    const bigData = new Array(1000000);

    return function inner() {
        console.log(bigData.length);
    };
}

const fn = outer();
```

`outer()` execute ho chuka hai, lekin `fn` ke paas `inner()` ka reference hai, aur `inner()` ko `bigData` chahiye.

Isliye:

```
fn
 ↓
inner()
 ↓
bigData
```

`fn` jab tak reachable hai, `bigData` bhi memory mein reh sakta hai.

Agar `fn` ki zarurat nahi:

JavaScript

```
fn = null;
```

(agar variable reassignment allowed ho) to GC eventually memory reclaim kar sakta hai.

6. Unnecessary Cache

JavaScript

```
const cache = {};

function addData(key, data) {
    cache[key] = data;
}
```

Agar cache mein continuously data add karte rahe:

```
cache
 ├── user1
 ├── user2
 ├── user3
 ├── user4
 ├── ...
 └── millions of objects
```

Aur old data kabhi remove nahi kiya, to memory continuously grow kar sakti hai.

Solution:

```javascript
delete cache["user1"];
```

Ya cache ko bounded/expiry-based rakho.

Garbage Collector ka connection

JavaScript mein **Garbage Collector (GC)** unreachable objects ko remove karta hai.

```javascript
let user = {
    name: "Tarun"
};

user = null;
```

Ab object ka koi reachable reference nahi hai.

Js code

Common Causes

Cause	Problem
Accidental globals	Object global scope mein retain
Event listeners	Unnecessary references
`setInterval()`	Continuous execution/reference
Closures	Outer data retain ho sakta hai
Large caches	Data continuously grow
DOM references	Removed DOM ko reference se hold karna